Bài tập lý thuyết tuần 3:
Nguyễn Minh Trường CNTT4
MSSV: 1250080218

1. Tên hệ quản trị CSDL NoSQL: Apache Cassandra
Apache Cassandra là hệ quản trị CSDL NoSQL dạng wide-column store, thiết kế để chạy phân tán, chịu lỗi tốt, mở rộng ngang (scale-out) dễ dàng, phù hợp dữ liệu lớn và hệ thống cần high availability.

2. Lịch sử hình thành:
Apache Cassandra được phát triển trong bối cảnh các hệ thống công nghệ thông tin hiện đại phải xử lý khối lượng dữ liệu rất lớn và yêu cầu hệ thống luôn hoạt động liên tục. Vào năm 2007, Facebook gặp nhiều khó khăn trong việc lưu trữ và tìm kiếm dữ liệu hộp thư của hàng trăm triệu người dùng. Các hệ quản trị cơ sở dữ liệu quan hệ truyền thống không còn đáp ứng được yêu cầu về tốc độ, khả năng mở rộng và độ sẵn sàng trong môi trường phân tán.
Để giải quyết vấn đề này, Facebook đã nghiên cứu và kết hợp hai công nghệ nổi bật trong lĩnh vực cơ sở dữ liệu phân tán là Amazon Dynamo và Google BigTable. Dynamo cung cấp mô hình hệ thống phân tán không có nút trung tâm, giúp tăng khả năng chịu lỗi và đảm bảo hệ thống luôn sẵn sàng. BigTable cung cấp mô hình lưu trữ dữ liệu theo dạng cột, cho phép xử lý dữ liệu với hiệu suất cao. Từ hai nền tảng này, Facebook đã phát triển một hệ quản trị cơ sở dữ liệu mới mang tên Cassandra nhằm phục vụ các ứng dụng cần xử lý dữ liệu ở quy mô lớn.
Năm 2008, Cassandra được công bố dưới dạng phần mềm mã nguồn mở và nhanh chóng thu hút sự quan tâm của cộng đồng công nghệ. Sau đó, dự án được chuyển giao cho Apache Software Foundation để tiếp tục phát triển và duy trì. Đến năm 2009, Cassandra chính thức trở thành một dự án cấp cao của Apache, đánh dấu sự trưởng thành và ổn định của hệ quản trị cơ sở dữ liệu này. Kể từ đó đến nay, Cassandra không ngừng được cải tiến và được ứng dụng rộng rãi trong nhiều hệ thống lớn trên thế giới.

3. Tác giả / tổ chức quản lý
Apache Cassandra ban đầu được phát triển bởi đội ngũ kỹ sư của Facebook, đứng đầu là Avinash Lakshman, một chuyên gia trong lĩnh vực hệ thống phân tán. Ông cũng là người đồng sáng tạo hệ thống Amazon Dynamo, một nền tảng quan trọng đã ảnh hưởng trực tiếp đến kiến trúc của Cassandra. Nhờ kinh nghiệm và kiến thức về các hệ thống phân tán quy mô lớn, nhóm phát triển đã xây dựng Cassandra với mục tiêu đạt được tính sẵn sàng cao, khả năng chịu lỗi tốt và khả năng mở rộng linh hoạt.
Sau khi Cassandra được phát hành dưới dạng mã nguồn mở, dự án đã được chuyển giao cho Apache Software Foundation, một tổ chức phi lợi nhuận chuyên quản lý và phát triển các dự án phần mềm mã nguồn mở. Hiện nay, Cassandra là một trong những dự án quan trọng của Apache và được phát triển bởi một cộng đồng toàn cầu gồm các lập trình viên, nhà nghiên cứu và kỹ sư phần mềm. Sự quản lý và phát triển liên tục của Apache đã giúp Cassandra duy trì được tính ổn định, độ tin cậy và khả năng mở rộng, đáp ứng tốt nhu cầu của các hệ thống dữ liệu lớn hiện đại.
4. Mô hình lưu trữ (Wide-column)
Cassandra lưu theo kiểu:
Keyspace: tương tự “database” (chứa cấu hình replication).
Table: giống bảng nhưng tối ưu theo truy vấn, không join.
Partition key: quyết định dữ liệu nằm ở node nào.

Clustering key: sắp xếp dữ liệu trong cùng partition (giúp query theo thời gian rất nhanh).

5. Ngôn ngữ thao tác dữ liệu: CQL

Cassandra dùng CQL (Cassandra Query Language), cú pháp gần giống SQL nhưng:
Không JOIN
Không hỗ trợ query tuỳ ý như SQL (đặc biệt là query không có partition key sẽ rất tốn hoặc bị chặn)

6. Cơ chế phân tán (phân mảnh + sao chép + truy vấn)

❖ Kiến trúc phân tán: Ring + Peer-to-peer:

Cassandra không có "master", mọi node ngang hàng (peer-to-peer).
Dữ liệu phân phối theo consistent hashing trên một "vòng" token (ring).
Node nào cũng có thể nhận request (node nhận request gọi là coordinator).
Phân mảnh dữ liệu (Partitioning):
Cassandra tính hash(partition_key) → ra token → ánh xạ token vào node chịu trách nhiệm.
Mỗi partition_key sẽ thuộc về một vùng token.

❖ Sao chép dữ liệu (Replication):

Cấu hình bằng replication_factor (RF).
Ví dụ RF=2: mỗi partition được lưu trên 2 node (1 node chính + 1 bản sao).
Nhất quán (Consistency) – "tunable":
Bạn chọn mức nhất quán khi đọc/ghi:
ONE: nhanh, có thể đọc "hơi cũ"
QUORUM: cân bằng
ALL: chắc chắn nhất nhưng chậm hơn

7. Mô phỏng lưu trữ & truy xuất phân tán

❖ Mô hình cluster mô phỏng

Giả sử có 3 node:
Node A: token range 0 → 33
Node B: token range 34 → 66
Node C: token range 67 → 99
Keyspace shopks đặt RF = 2.
Quy tắc (đơn giản hóa):
Token thuộc node nào → node đó là primary replica
Bản sao thứ 2 nằm ở node kế tiếp theo chiều kim đồng hồ (clockwise)

❖ Dữ liệu mẫu:

Giả sử ta có 3 khách hàng:

customer_id	hash(token) giả sử
C001	20
C002	50
C003	85

Phân phối dữ liệu theo token + RF=2
C001 token=20 → thuộc Node A (primary), replica trên Node B
C002 token=50 → thuộc Node B (primary), replica trên Node C
C003 token=85 → thuộc Node C (primary), replica trên Node A
Tóm tắt:

Customer	Primary

C001	A
C002	B
C003	C

❖ Mô phỏng thao tác GHI (WRITE) phân tán:

Yêu cầu:

Thêm 1 đơn hàng của khách C002.

```
INSERT INTO orders_by_customer (customer_id, order_time, order_id, total, status)
VALUES ('C002', toTimestamp(now()), 'OD9001', 1250000, 'PAID');
```

Luồng xử lý (mô phỏng)

Client gửi request đến Node A (bất kỳ node nào cũng được).

→ Node A trở thành coordinator.

Coordinator tính token của partition_key = hash('C002') = 50

→ Primary của token=50 là Node B, replica là Node C.

Với consistency QUORUM (RF=2 → quorum=2):

Coordinator gửi ghi đến Node B và Node C

Cần cả 2 node xác nhận → mới trả "SUCCESS" về client.

Kết quả: dữ liệu đơn hàng C002 tồn tại trên B và C.

❖ Mô phỏng thao tác ĐỌC (READ) phân tán:

Truy vấn lịch sử mua hàng của C002 (10 đơn gần nhất)

```
SELECT order_id, order_time, total, status
FROM orders_by_customer
WHERE customer_id = 'C002'
LIMIT 10;
```

Luồng xử lý:

Client gửi query đến Node C → Node C làm coordinator.

Coordinator tính token(customer_id='C002') = 50 → dữ liệu nằm ở Node B & C.

Với consistency ONE:

Coordinator chỉ cần đọc từ 1 replica (ưu tiên local replica nếu có).

Vì coordinator là Node C và Node C đang giữ replica của C002 → đọc tại chỗ rất nhanh.

❖ Mô phỏng truy vấn "khó" và cách thiết kế để phân tán hiệu quả:

Bài toán: tìm tất cả đơn hàng theo status='PAID'

Nếu bạn viết:

```
SELECT * FROM orders_by_customer WHERE status='PAID';
```

Cassandra sẽ không cho (hoặc rất tốn) vì status không phải partition key → sẽ phải scan nhiều node.

Cách làm đúng (mô hình hoá theo truy vấn)

Tạo bảng phụ (denormalization) để truy vấn theo status:

```
CREATE TABLE orders_by_status (
  status text,
  order_time timestamp,
  customer_id text,
  order_id text,
  total double,
  PRIMARY KEY ((status), order_time, order_id)
) WITH CLUSTERING ORDER BY (order_time DESC);
```

Khi ghi đơn hàng, ghi 2 bảng:

```
BEGIN BATCH
INSERT INTO orders_by_customer (...) VALUES (...);
INSERT INTO orders_by_status   (...) VALUES (...);
```

APPLY BATCH;
Bây giờ query theo status:
SELECT order_id, customer_id, total
FROM orders_by_status
WHERE status='PAID'
LIMIT 20;
Token lúc này dựa trên status → partition nằm trên một vài node thay vì quét toàn cluster.

8. Kết luận
Cassandra phù hợp hệ thống cần phân tán, chịu lỗi, tốc độ cao, ghi nhiều.
Dữ liệu được phân tán theo partition key → token → node, sao chép theo RF.
Truy vấn phân tán hiệu quả khi thiết kế bảng đúng theo query-first (tạo các bảng chuyên dụng thay vì join).